

Concurrency Control Techniques

▼ Intro

This chapter discusses various **concurrency control techniques** used to ensure **isolation** (noninterference) among concurrently executing transactions in a database.

The main goal of these techniques is to ensure **serializability** — meaning that the outcome of concurrent transactions is the same as if they were executed one after another (serially)

▼ Two-Phase Locking Techniques for Concurrency Control

◆ Purpose

- **Goal:** Control the **concurrent execution** of transactions to ensure **isolation** and **serializability**.
 - **Method:** Use **locks** to manage and synchronize access to data items in a database.
-

◆ What is a Lock?

- A **lock** is a **variable** associated with each **data item**.
 - It represents the **status** of that item — i.e., whether it's free or currently being used by a transaction.
 - Locks prevent conflicts by ensuring **only one transaction** can modify (or sometimes read) a data item at a time.
 - Typically, **every data item** in a database has **one lock**.
-

◆ How Locks Work

- Locks are a **synchronization mechanism**:
They coordinate access to shared data when multiple transactions run at the same time.

If the simple binary locking scheme described here is used, every transaction must obey the following rules:

1. A transaction T must issue the operation $\text{lock_item}(X)$ before any $\text{read_item}(X)$ or $\text{write_item}(X)$ operations are performed in T .
2. A transaction T must issue the operation $\text{unlock_item}(X)$ after all $\text{read_item}(X)$ and $\text{write_item}(X)$ operations are completed in T .
3. A transaction T will not issue a $\text{lock_item}(X)$ operation if it already holds the lock on item X .¹
4. A transaction T will not issue an $\text{unlock_item}(X)$ operation unless it already holds the lock on item X .

▼ Binary Locks

```
lock_item(X):
B:  if LOCK(X) = 0          (* item is unlocked *)
    then LOCK(X) ← 1      (* lock the item *)
    else
        begin
            wait (until LOCK(X) = 0
                and the lock manager wakes up the transaction);
            go to B
        end;
unlock_item(X):
    LOCK(X) ← 0;          (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
```

Figure 22.1
Lock and unlock operations for binary locks.

If the simple binary locking scheme described here is used, every transaction must obey the following rules:

1. A transaction T must issue the operation $\text{lock_item}(X)$ before any $\text{read_item}(X)$ or $\text{write_item}(X)$ operations are performed in T .
2. A transaction T must issue the operation $\text{unlock_item}(X)$ after all $\text{read_item}(X)$ and $\text{write_item}(X)$ operations are completed in T .
3. A transaction T will not issue a $\text{lock_item}(X)$ operation if it already holds the lock on item X .¹
4. A transaction T will not issue an $\text{unlock_item}(X)$ operation unless it already holds the lock on item X .

1. Purpose

Locks are used in **concurrency control** to prevent conflicting operations when multiple transactions access the same data items.

Each data item in the database is associated with a **lock variable** that indicates whether the item is free or in use.

◆ 2. Types of Locks

● Binary Locks

- **Simplest form** of locking (used for understanding; not practical in real systems).
- Each lock can have **only two states**:
 - `0 (unlocked)` — item is available.
 - `1 (locked)` — item is in use and cannot be accessed by others.

► Basic Operations

1. `lock_item(X)`
 - Checks if `LOCK(X) = 0`.
 - If yes → sets `LOCK(X) = 1` and grants access.
 - If no → the transaction **waits** until `LOCK(X)` becomes `0`.
2. `unlock_item(X)`
 - Sets `LOCK(X) = 0`.
 - Wakes up **one waiting transaction** (if any).

✚ *These operations must be atomic (indivisible)* — no other process can interrupt them while executing.

If several transactions want the same item, they're placed in a **waiting queue** for that item.

◆ 3. System Lock Table

- The DBMS keeps a **Lock Table** that stores information about **currently locked items**.
- Each entry may include:

```
<Data_item_name, LOCK, Locking_transaction>  
+ Queue of waiting transactions
```

- Items **not** listed in the lock table are **unlocked**.
- The **Lock Manager** subsystem handles:
 - Maintaining the lock table
 - Granting/releasing locks
 - Managing waiting queues

◆ 4. Rules for Using Binary Locks

Every transaction **must follow these rules**:

1. **Before Accessing:**

A transaction **must lock** an item (`lock_item(X)`) **before** reading or writing it.

2. **After Finishing:**

It must **unlock** the item (`unlock_item(X)`) **after** all read/write operations are done.

3. **No Double Locking:**

A transaction cannot issue another `lock_item(X)` if it already holds the lock on `X`.

(This can be relaxed if the lock operation checks ownership.)

4. **No Unlock Without Ownership:**

A transaction can only unlock an item if it **currently holds** its lock.

♦ **5. Effect of Binary Locking**

- Only **one transaction at a time** can hold a lock on a data item.
- Therefore, **no two transactions** can access the same item **concurrently**.
- This guarantees **mutual exclusion**, but also severely limits concurrency — which is why **binary locks are not used in practice**.

▼ Shared/Exclusive (or Read/Write) Locks

```
read_lock(X):
B:  if LOCK(X) = "unlocked"
      then begin LOCK(X) ← "read-locked";
           no_of_reads(X) ← 1
           end
      else if LOCK(X) = "read-locked"
           then no_of_reads(X) ← no_of_reads(X) + 1
      else begin
           wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
           go to B
           end;
write_lock(X):
B:  if LOCK(X) = "unlocked"
      then LOCK(X) ← "write-locked"
      else begin
           wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
           go to B
           end;
unlock (X):
      if LOCK(X) = "write-locked"
          then begin LOCK(X) ← "unlocked";
               wakeup one of the waiting transactions, if any
               end
      else if LOCK(X) = "read-locked"
          then begin
               no_of_reads(X) ← no_of_reads(X) - 1;
               if no_of_reads(X) = 0
                   then begin LOCK(X) = "unlocked";
                        wakeup one of the waiting transactions, if any
                        end
               end;
      end;
```

Figure 22.2
Locking and unlocking
operations for two-
mode (read-write or
shared-exclusive)
locks.

When we use the shared/exclusive locking scheme, the system must enforce the following rules:

1. A transaction T must issue the operation $\text{read_lock}(X)$ or $\text{write_lock}(X)$ before any $\text{read_item}(X)$ operation is performed in T .
2. A transaction T must issue the operation $\text{write_lock}(X)$ before any $\text{write_item}(X)$ operation is performed in T .

3. A transaction T must issue the operation $\text{unlock}(X)$ after all $\text{read_item}(X)$ and $\text{write_item}(X)$ operations are completed in T .³
4. A transaction T will not issue a $\text{read_lock}(X)$ operation if it already holds a read (shared) lock or a write (exclusive) lock on item X . This rule may be relaxed, as we discuss shortly.

this rule may be relaxed to allow a transaction to unlock an item, then lock it again later.

2 Chapter 22 Concurrency Control Techniques

5. A transaction T will not issue a $\text{write_lock}(X)$ operation if it already holds a read (shared) lock or write (exclusive) lock on item X . This rule may also be relaxed, as we discuss shortly.
6. A transaction T will not issue an $\text{unlock}(X)$ operation unless it already holds a read (shared) lock or a write (exclusive) lock on item X .

♦ 1. Why Binary Locks Are Too Restrictive

- Binary locks allow **only one transaction at a time** to access a data item.
- Problem: **Multiple transactions can safely read the same item simultaneously**, so binary locks unnecessarily block concurrent reads.
- Solution: Use **multiple-mode locks** (shared/exclusive locks).

♦ 2. Concept

- **Shared (Read) Lock:**
 - Also called **read-locked** or **share-locked**.
 - Multiple transactions can hold a shared lock on the same item.
 - Allows **concurrent reads**.
- **Exclusive (Write) Lock:**
 - Also called **write-locked**.
 - Only **one transaction** can hold a write lock on an item.
 - Prevents both reads and writes by other transactions.
- **Unlocked:**

- No transaction holds any lock on the item.

♦ 3. Lock Operations

1. `read_lock(X)`

- If `X` is unlocked → set `LOCK(X) = read-locked`, `No_of_reads = 1`.
- If `X` already read-locked → increment `No_of_reads`.
- If `X` write-locked → transaction **waits**.

2. `write_lock(X)`

- If `X` is unlocked → set `LOCK(X) = write-locked`.
- If `X` locked (read or write) → transaction **waits**.

3. `unlock(X)`

- If `X` write-locked → unlock and wake up **one waiting transaction**.
- If `X` read-locked → decrement `No_of_reads`.
 - If `No_of_reads = 0` → unlock and wake up one waiting transaction.

♦ All operations are atomic — no interleaving allowed while a lock/unlock operation is executing.

♦ 4. Lock Table Structure

Each **locked item** in the lock table may store:

```
<Data_item_name, LOCK (read/write), No_of_reads, Locking_transaction(s)>
```

- **No_of_reads**: Number of transactions holding a shared lock.
- **Locking_transaction(s)**: Either a single transaction (write lock) or a list (shared locks).

♦ 5. Rules for Transactions

1. Must issue `read_lock(X)` **before reading**.
2. Must issue `write_lock(X)` **before writing**.
3. Must issue `unlock(X)` **after completing operations**.
4. Cannot issue a lock if already holding a lock on the same item (relaxable).
5. Cannot unlock unless currently holding a lock.

* Problems in S/X locking

- T_1
 $\frac{S(A)}{R(A)}$
 $U(A)$
 $R-R$
 (A)
- T_2
 $\frac{X(A)}{R(A)}$
 $W(A)$
 $U(A)$
 (A)
- $\downarrow$
 $grant$
- S
 X
- | | Request | |
|---|---------|----|
| | S | X |
| S | Yes | No |
| X | No | No |
- (A)
- (A)
 $R(A)$
 $W(A)$
 $R(A)$

- ◆ 1. Purpose

- ## ◆ 2. Types of Lock Conversion

- A transaction T holds a **read (shared) lock** on item X.
- Later, T wants a **write (exclusive) lock**.
- **Condition:** Upgrade is allowed **only if T is the only transaction holding a read lock** on X; otherwise, T must wait.

- A transaction T holds a **write lock** on X.
- Later, it may **downgrade to a read lock**.
- Useful when the exclusive access is no longer needed but T still wants to read the item.

3. Implementation Notes

- 8

- The algorithms for `read_lock(X)` and `write_lock(X)` must be modified to support upgrading and downgrading.

♦ 4. Why Locking Alone Doesn't Guarantee Serializability

- Even with proper use of binary or read/write locks, **non-serializable schedules** can occur if **locks are released too early**.
- Example:
 - Transaction T1 unlocks item Y too soon.
 - Transaction T2 accesses X concurrently.
 - The resulting schedule may **not be serializable**, producing **incorrect results**.
-  **Solution:** Use a strict protocol for the **position of lock and unlock operations**.

▼ Explanation

♦ Transactions

T1:

```
read Y
unlock Y
write X = X + Y
```

T2:

```
read X
write Y = Y + X
```

♦ Step 1: Interleaved schedule (early unlock)

Let's assume **T1 unlocks Y too early**, allowing T2 to proceed:

1. **T1 reads Y = 30**
2. **T1 unlocks Y** → T2 can access Y
3. **T2 reads X = 20**
4. **T2 writes Y = Y + X = 30 + 20 = 50**
5. **T1 writes X = X + Y = 20 + 30 = 50**

 **Result:** X = 50, Y = 50

♦ Step 2: Compare with serial schedules

1. T1 then T2 (serial):

- T1: $X = 20 + 30 = 50$
- T2: $Y = 30 + 50 = 80$
- ☒ Final: $X=50, Y=80$

2. T2 then T1 (serial):

- T2: $Y = 30 + 20 = 50$
- T1: $X = 20 + 50 = 70$
- ☒ Final: $X=70, Y=50$

♦ Step 3: Check the interleaved schedule

- Interleaved schedule: $X = 50, Y = 50$
- Compare with serial schedules:
 - T1 → T2: $X=50, Y=80$ → does **not match**
 - T2 → T1: $X=70, Y=50$ → does **not match**

☒ **Conclusion:** Non-serializable.

▼ Intuition

Intuition Behind the Problem

1. Serializability is about “as if transactions happened one after another.”

- Even if operations are interleaved, the final result should be **the same as some serial order**.

2. Early unlock breaks this “as if” property.

- When T1 unlocks Y too soon:
 - Other transactions (like T2) can **write to Y** before T1 finishes using it.
- Even though T1 already **read Y**, it still **depends on Y's value** when it updates X.

3. Dependencies matter, not just reads.

- T1's update $X := X + Y$ depends on **the value of Y at the time T1 read it**.
 - If T2 modifies Y in between, **the “story” of the transactions gets scrambled**.
 - Now, **X and Y are not consistent with any serial execution**, because the operations' effects overlap in a conflicting way.
-

♦ A Simple Analogy

Imagine **two people sharing a notebook**:

- **Person A** (T1) writes: "X = old X + Y"
- **Person B** (T2) writes: "Y = old Y + X"

If A finishes reading Y but lets B change it **before updating X**, it's like A's calculation is **based on outdated information**, but B has already changed the reality.

- The final numbers no longer match any order where one person fully goes first — that's why **it's non-serializable**

▼ Guaranteeing Serializability by Two-Phase Locking

Chapter 22 Concurrency Control Techniques

Figure 22.4

Transactions T_1' and T_2' , which are the same as T_1 and T_2 in Figure 22.3, but follow the two-phase locking protocol. Note that they can produce a deadlock.

T_1'	T_2'
<code>read_lock(Y);</code> <code>read_item(Y);</code> <code>write_lock(X);</code> <code>unlock(Y)</code> <code>read_item(X);</code> <code>X := X + Y;</code> <code>write_item(X);</code> <code>unlock(X);</code>	<code>read_lock(X);</code> <code>read_item(X);</code> <code>write_lock(Y);</code> <code>unlock(X)</code> <code>read_item(Y);</code> <code>Y := X + Y;</code> <code>write_item(Y);</code> <code>unlock(Y);</code>

Two-Phase Locking (2PL) Idea

A transaction is divided into **two phases**:

1. Growing (Expanding) Phase

- The transaction can **acquire new locks** (read or write).
- **No locks are released** in this phase.

2. Shrinking Phase

- The transaction can **release locks**.
- **No new locks are acquired** in this phase.

This ensures that dependencies between transactions are respected, preventing non-serializable schedules.

♦ Why this guarantees serializability

- If **all transactions follow 2PL**, locks create a **partial ordering** of operations.
- This prevents the "early unlock" problem we discussed:

- No transaction can **release a lock too soon** and let others interfere before it finishes using all its data items.
- Therefore, the final schedule is **equivalent to some serial order**, guaranteeing correctness.

▼ Example

Let's walk **step by step** through the previous example (**T1 and T2**) and see **exactly how 2PL fixes the problem**.

♦ Initial Values

$X = 20, Y = 30$

♦ Transactions

T1:

1. read Y
2. write $X = X + Y$

T2:

1. read X
2. write $Y = Y + X$

♦ Problem Without 2PL

- T1 **unlocks Y too early**, so T2 can modify Y before T1 writes X.
- Interleaved schedule:

Step	Transaction	Operation	X	Y
1	T1	read Y	20	30
2	T1	unlock Y	20	30
3	T2	read X	20	30
4	T2	write $Y = Y + X$	20	50
5	T1	write $X = X + Y$	50	50

- **Final state:** $X=50, Y=50 \rightarrow$ **non-serializable**, as explained earlier.

♦ Step 1: Apply 2PL

Rule:

- Growing phase $\rightarrow$ acquire all locks needed.

- Shrinking phase → release locks, no new locks.

We rewrite T1 and T2 following **2PL**:

T1:

```
read_lock(Y)
read_item(Y)
write_lock(X)
write_item(X := X + Y)
unlock(Y)
unlock(X)
```

T2:

```
read_lock(X)
read_item(X)
write_lock(Y)
write_item(Y := Y + X)
unlock(X)
unlock(Y)
```

Locks are acquired before first unlock → satisfies 2PL.

♦ Step 2: Interleaving With 2PL

- **T1 acquires lock on Y first.**
- T2 wants to write Y, but Y is locked by T1 → **T2 waits.**
- T1 acquires lock on X → now T1 has **all locks.**
- T1 performs write operations.
- T1 releases locks → **T2 can proceed.**

Step	Transaction	Operation	X	Y
1	T1	read_lock(Y)	20	30
2	T1	read Y	20	30
3	T1	write_lock(X)	20	30
4	T1	write X = X+Y	50	30
5	T1	unlock(Y)	50	30
6	T1	unlock(X)	50	30
7	T2	read_lock(X)	50	30
8	T2	read X	50	30

Step	Transaction	Operation	X	Y
9	T2	write_lock(Y)	50	30
10	T2	write Y = Y+X	50	80
11	T2	unlock(X)	50	80
12	T2	unlock(Y)	50	80

✓ **Final state:** X=50, Y=80 → matches **T1 → T2 serial execution**.

♦ Step 3: Key Intuition

- T2 **cannot access Y before T1 finishes X**, because **T1 is still holding Y**.
- **Dependencies are preserved** → the schedule is serializable.
- No intermediate inconsistent values like X=50, Y=50 occur.

▼ Deadlock

A **deadlock** occurs in a database (or any concurrent system) when **two or more transactions are waiting indefinitely for resources held by each other**, so none of them can proceed.

Think of it like this:

Example Scenario

- **Transaction T1** holds a lock on item X and wants to lock item Y.
- **Transaction T2** holds a lock on item Y and wants to lock item X.
- Both transactions **wait forever** because each is waiting for the other to release a lock.

```
T1: Lock(X) → wants Lock(Y) → waiting
T2: Lock(Y) → wants Lock(X) → waiting
```

Neither transaction can continue → **deadlock**.

Key Points

1. **Deadlock happens only with multiple transactions** competing for shared resources.
2. **Resources are usually locks on database items**, but could be other things like memory or CPU.
3. **Two conditions are necessary for a deadlock:**
 - **Mutual exclusion:** Each resource can be held by only one transaction at a time.
 - **Circular wait:** A chain of transactions exists where each transaction waits for a resource held by the next transaction in the chain.

How DBMS handles deadlocks

1. **Detection:** Periodically check for cycles in the “wait-for” graph of transactions.
2. **Resolution:** Abort (roll back) one of the transactions in the cycle to break the deadlock.
3. **Prevention (optional):** Use strategies like **Conservative 2PL**, where transactions lock everything they need in advance, so deadlocks cannot occur.

▼ Variations of 2PL

◆ 1. Variations of 2PL

There are several forms of 2PL, each with different rules and trade-offs:

a) Basic 2PL

- This is the standard 2PL we discussed before.
 - **Rule:** All locks must be acquired before any unlock occurs (growing phase first, then shrinking).
 - Guarantees **serializability**.
 - **Problem:** Deadlocks can occur if two transactions wait for each other's locks.
-

b) Conservative (Static) 2PL

- **Idea:** Prevent deadlocks by **locking all items before the transaction starts**.
 - How it works:
 1. The transaction **predeclares its read-set and write-set** (all items it will access).
 2. If **all items can be locked**, it proceeds.
 3. If **any item is unavailable**, the transaction waits and locks **nothing**.
 -  **Advantage:** Deadlock-free (because a transaction never waits while holding some locks).
 -  **Disadvantage:** Hard to use in practice, because you often **don't know all items in advance**.
-

c) Strict 2PL

- **Idea:** Ensure recoverable schedules by **holding write locks until commit/abort**.
- Rule: A transaction **does not release exclusive (write) locks** until it finishes.
-  Guarantees **strict schedules**, meaning no other transaction can read/write uncommitted data.

- ❌ Still **not deadlock-free** (two transactions can still wait for each other).
-

d) Rigorous 2PL

- **Idea:** Even stricter than strict 2PL.
- Rule: A transaction **holds all locks (read and write) until commit/abort**.
- ✅ Guarantees strict schedules and easier to implement (locks released automatically at the end).
- **Difference from conservative 2PL:**
 - Conservative: must lock everything **before starting** → shrinking phase starts immediately.
 - Rigorous: can acquire locks as needed, but **never unlocks until commit/abort** → growing phase continues until the end.

▼ Deadlock Prevention Protocols

Deadlock Recap

A **deadlock** happens when transactions are waiting for each other in a circular way:

- **T1** holds a lock on X, wants Y
- **T2** holds a lock on Y, wants X
- Both wait forever → **deadlock**

This can be visualized with a **wait-for graph**, where nodes are transactions and edges point from a waiting transaction to the transaction holding the lock it wants.

Deadlock Prevention Protocols

Deadlocks can be prevented by carefully controlling **which transactions wait and which abort**.

1. Conservative Two-Phase Locking

- **Idea:** Lock all items a transaction will access **before it starts**.
- If even one lock cannot be acquired, the transaction waits and retries.
- **Pros:** Deadlock-free.
- **Cons:** Limits concurrency; impractical if you can't know all items in advance.

2. Ordering Items

- Assign a fixed order to all items.
- Transactions must acquire locks **in that order**.

- Prevents circular wait → deadlock-free.
- **Cons:** Hard to manage in dynamic databases.

3. Timestamp-Based Schemes

Each transaction gets a **timestamp** $TS(T)$ based on its start time (older → smaller timestamp).

Two main protocols:

1. Wait-Die

- If **older transaction** requests a locked item, it waits.
- If **younger transaction** requests a locked item, it aborts and restarts.
- Prevents deadlocks because only older transactions can wait → no cycles.

2. Wound-Wait

- If **older transaction** requests a locked item held by a younger transaction, it **preempts/aborts** the younger transaction (wounds it).
- If **younger transaction** requests a locked item held by an older transaction, it waits.
- Deadlock-free: only younger transactions wait → no cycles.

Both methods aim to abort the “younger” transaction in a potential deadlock scenario, minimizing wasted work.

decision about what to do with a transaction involved in a possible deadlock situation: Should it be blocked and made to wait or should it be aborted, or should the transaction preempt and abort another transaction? Some of these techniques use the concept of **transaction timestamp** $TS(T)$, which is a unique identifier assigned to each transaction. The timestamps are typically based on the order in which transactions are started; hence, if transaction T_1 starts before transaction T_2 , then $TS(T_1) < TS(T_2)$. Notice that the *older* transaction (which starts first) has the *smaller* timestamp value. Two schemes that prevent deadlock are called *wait-die* and *wound-wait*. Suppose that transaction T_i tries to lock an item X but is not able to because X is locked by some other transaction T_j with a conflicting lock. The rules followed by these schemes are:

- **Wait-die.** If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) T_i is allowed to wait; otherwise (T_i younger than T_j) abort T_i (T_i dies) and restart it later *with the same timestamp*.
- **Wound-wait.** If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) abort T_j (T_i wounds T_j) and restart it later *with the same timestamp*; otherwise (T_i younger than T_j) T_i is allowed to wait.

In wait-die, an older transaction is allowed to *wait for a younger transaction*, whereas a younger transaction requesting an item held by an older transaction is aborted

4. Non-Timestamp Protocols

- **No Waiting (NW):**

If a transaction cannot get a lock, it immediately aborts and restarts.

- **Pros:** Deadlock-free.
- **Cons:** Can cause unnecessary aborts.
- **Cautious Waiting (CW):**

A transaction waits only if the transaction holding the lock is **not itself blocked**.

- Ensures a total order on waiting transactions → deadlock-free.
- Reduces unnecessary aborts compared to NW.

Intuition

Deadlocks occur because of **circular waits**. All these protocols break the cycle by either:

1. Preventing circular waits (ordering locks, CW)
2. Killing one transaction in the potential cycle (wait-die, wound-wait)
3. Locking all items in advance (conservative 2PL)

Step 2: Impose a fixed order on items

Suppose we assign a **global order** to items:

- $X = 1$
- $Y = 2$
- $Z = 3$

Rule: Every transaction must acquire locks in **ascending order** ($1 \rightarrow 2 \rightarrow 3$).

Step 3: See how circular wait is impossible

- T1 wants X then Y → order $1 \rightarrow 2$ ✅ allowed
- T2 wants Y then X → order $2 \rightarrow 1$ ❌ not allowed

Because T2 must acquire locks in ascending order, it **cannot request Y first and then X**.

Thus, the situation where:

- T1 holds X, waits for Y
- T2 holds Y, waits for X

cannot happen, because T2 would have to request X **before** Y, not after.

No circular wait → **deadlock is impossible**.

▼ Deadlock Detection

1. Deadlock Detection

Instead of preventing deadlocks, the system **lets them happen** and then detects them:

- The system maintains a **wait-for graph**:
 - Nodes = transactions currently running.

- Edge $T_i \rightarrow T_j$ = T_i is waiting for an item locked by T_j .
- **Cycle in the graph = deadlock.**
- Example: T_1 waits for X locked by T_2 , T_2 waits for Y locked by $T_1 \rightarrow$ cycle $\rightarrow$ deadlock.

Key questions:

- When should the system check for cycles?
Checking after every lock request is expensive.
Alternatives: check periodically or based on transaction wait times.

Victim selection:

- If deadlock occurs, some transaction(s) must be aborted to break it.
- Choose transactions that:
 - Haven't done much work yet (younger transactions).
 - Avoid aborting transactions that already performed many updates.

2. Timeouts

- Simpler approach.
- If a transaction waits too long for a lock, assume a deadlock and **abort it**.
- Doesn't guarantee deadlock detection but has low overhead.

3. Starvation

Starvation happens when a transaction **never gets a chance** to acquire a lock:

- Caused by **unfair waiting schemes** or repeated victim selection.
- Solutions:
 - **Fair queue**: First-come-first-served.
 - **Priority aging**: Increase priority the longer a transaction waits.
 - **Careful victim selection**: Avoid repeatedly aborting the same transaction.
 - Wait-die and wound-wait schemes naturally avoid starvation by respecting transaction timestamps.

Intuition:

- Deadlock = transactions "blocking each other in a loop."
- Deadlock detection = spot the loop, then break it by aborting someone.
- Starvation = a transaction is **always last in line**, never making progress.

▼ Concurrency Control Based on Timestamp Ordering

1. Motivation

- With **2PL (locking)**, serializability is guaranteed, but:
 - Transactions may **wait** if items are locked.
 - **Deadlocks** can occur.
- Timestamp-based concurrency control avoids locks **entirely**, so **no deadlocks**.

2. Timestamps

- Each transaction T gets a **unique timestamp** $TS(T)$ when it starts.
 - Can be generated using a **counter** (1, 2, 3...) or **system clock**.
 - Think of the timestamp as the transaction's "start time."
- This timestamp represents the **order in which transactions should appear in the serial schedule**.

3. Timestamp Ordering (TO) Principle

- Transactions are executed in a way that guarantees the **final schedule is equivalent to the serial order of timestamps**.
- Key idea: if $T1$ has $TS(T1) < TS(T2)$, then $T1$ should appear **before** $T2$ in the serializable schedule.

4. Read and Write Timestamps for Items

For each database item X , maintain two timestamps:

1. **read_TS(X)**: The **largest timestamp** among all transactions that have successfully read X .
 - Represents the most recent "reader" of X .
2. **write_TS(X)**: The **largest timestamp** among all transactions that have successfully written X .
 - Represents the most recent "writer" of X .

1. Read Operation (T wants to read X)

- Rule: T can read X only if $TS(T) \geq write_TS(X)$
- Explanation:
 - If a newer transaction has already written X ($write_TS(X) > TS(T)$), then T is "too old" to read X.
 - Reading would violate the timestamp order, so T must abort.

✓ So the read condition is:

$$TS(T) \geq write_TS(X)$$

- After a successful read:

$$read_TS(X) := \max(read_TS(X), TS(T))$$

2. Write Operation (T wants to write X)

- Rule: T can write X only if $TS(T) \geq read_TS(X)$ AND $TS(T) \geq write_TS(X)$
- Explanation:
 - If a newer transaction has read X ($read_TS(X) > TS(T)$), T writing now would overwrite a value that a younger transaction has already read, violating serializability.
 - If a newer transaction has already written X ($write_TS(X) > TS(T)$), T is "too old" to write.

✓ So the write condition is:

$$TS(T) \geq read_TS(X) \quad \text{and} \quad TS(T) \geq write_TS(X)$$

- After a successful write:

$$write_TS(X) := TS(T)$$

▼ Rules

1. Basic Timestamp Ordering (Basic TO)

Idea:

- Each transaction has a unique timestamp $TS(T)$.
- The database keeps two timestamps for each item X :
 1. $read_TS(X)$: timestamp of the **latest transaction that read X**.
 2. $write_TS(X)$: timestamp of the **latest transaction that wrote X**.

Rules:

a) Read operation: `read_item(X)` by T

- Allowed **only if** $TS(T) \geq write_TS(X)$
 - Why? Because if a newer transaction has already written X, T reading X now would violate timestamp order.
- If allowed: update `read_TS(X)` to `max(read_TS(X), TS(T))`.
- If violated: **abort T**.

b) Write operation: `write_item(X)` by T

- Allowed **only if**:
$$TS(T) \geq read_TS(X) \quad \text{AND} \quad TS(T) \geq write_TS(X)$$
 - If violated: some younger transaction already read or wrote X → **abort T**.
- If allowed: update `write_TS(X)` = `TS(T)`.

✓ Result:

- Schedules are **conflict serializable**.
- **No deadlocks**, because transactions never wait; they are aborted instead.
- **Problem**: cascading aborts may occur, because rolling back one transaction may require rolling back others that depended on it.



2. Strict Timestamp Ordering (Strict TO)

Problem with basic TO:

- Basic TO may produce **non-recoverable schedules** (cascading rollbacks).

Solution:

- Strict TO **delays reads/writes** on an item X if a transaction with smaller timestamp wrote X but has not yet committed.
- Essentially, treat uncommitted writes like locks until commit/abort.

Effect:

- Produces **strict schedules** → no cascading rollbacks.
- Still **deadlock-free**, because transactions only wait for older transactions (timestamp-based).

3. Thomas's Write Rule

Observation:

- Sometimes a write by an "older" transaction T may be **obsolete**, because a younger transaction already wrote X.

Rule modification:

1. If `read_TS(X) > TS(T)` → **abort T**.
 - Older transaction reading a value already read by a younger transaction violates timestamp order.
2. If `write_TS(X) > TS(T)` → **ignore T's write**, but continue.
 - Older transaction writing a value that has already been written by a younger transaction is **obsolete**.
3. Otherwise → execute write and update `write_TS(X)`.

Effect:

- Fewer aborts compared to basic TO.
- May **not be fully conflict serializable**, but still **preserves** a logical timestamp order.



▼ MVCC

1. Idea of Multiversion Concurrency Control (MVCC)

- Instead of maintaining only **one value** per data item, the database keeps **multiple versions** of each item.
- Each version records:
 1. The value of the item at the time of some transaction.
 2. Timestamps: `write_TS` (when it was written) and `read_TS` (latest transaction that read it).
- **Benefit:**
 - Some read operations that would normally conflict under 2PL or basic timestamp ordering can now read an **older version** instead of being aborted.
 - This improves concurrency—more transactions can proceed simultaneously.
- **Cost:**

- More storage is required.
- Useful in temporal databases or for auditing changes anyway.

2. MVCC based on Timestamp Ordering

Suppose we have **item X** with multiple versions: $x_1, x_2, \dots, x_k$.

Each version x_i has:

- $write_TS(x_i)$: timestamp of the transaction that created it.
- $read_TS(x_i)$: timestamp of the latest transaction that read it.

a) Writing a new version

When transaction **T** wants to write x :

1. Find the version x_i of **X** with:
 - The **largest** $write_TS(x_i) \leq TS(T)$.
 - This is the most recent version that **T** is “allowed” to see.
2. Check the read conflict:
 - If $read_TS(x_i) > TS(T) \rightarrow$ abort **T**.
 - Why? Some **younger transaction** already read x_i , so **T** writing now would violate serializability.
 - Otherwise, create a **new version** x_j with:

```
write_TS(x_j) = TS(T)
read_TS(x_j) = TS(T)
```

- **T** writes its changes to a **new version** instead of overwriting the old one.

b) Reading a version

When transaction **T** wants to read x :

1. Find the version x_i with:
 - The **largest** $write_TS(x_i) \leq TS(T)$.
 - This ensures **T** reads the **latest committed version** that existed before **T** started.
2. Return the value of x_i to **T**.
3. Update $read_TS(x_i) = \max(read_TS(x_i), TS(T))$

✓ **Result:**

- The read **always succeeds** because T can always find an older version that fits its timestamp.
- No read operation will abort in this scheme, unlike basic timestamp ordering.

3. How MVCC improves concurrency

- Multiple transactions can **read old versions** simultaneously.
- Writers don't block readers—they create **new versions**.
- Serializability is maintained using timestamps to order versions.

Example Scenario:

- **T1** writes X=10 at TS=1 → creates X1 (write_TS=1).
- **T2** reads X at TS=2 → reads X1.
- **T3** writes X at TS=3 → creates X2 (write_TS=3).
- **T4** reads X at TS=4 → reads X2 (latest ≤ 4).

All reads succeed, no waiting, serializability is maintained.

4. Key Rules Recap

Operation	Rule
Read X by T	Return version Xi with largest $\text{write_TS}(X_i) \leq \text{TS}(T)$. Update $\text{read_TS}(X_i) = \max(\text{read_TS}(X_i), \text{TS}(T))$.
Write X by T	Check version Xi with largest $\text{write_TS}(X_i) \leq \text{TS}(T)$. If $\text{read_TS}(X_i) > \text{TS}(T)$ → abort. Else, create new version Xj with $\text{write_TS}(X_j) = \text{TS}(T)$ and $\text{read_TS}(X_j) = \text{TS}(T)$.

✓ Summary of Advantages

1. High concurrency: readers never block writers.
2. Readers can always read some version → fewer aborts.
3. Serializability is maintained based on timestamp ordering.

✗ Drawbacks

1. More storage required for multiple versions.
2. Cascading rollbacks are still possible if writers are aborted.
3. Old versions may need garbage collection to save space.

▼ Multiversion Two-Phase Locking (MV2PL)

(a)		Read	Write
Read		Yes	No
Write		No	No

(b)		Read	Write	Certify
Read		Yes	Yes	No
Write		Yes	No	No
Certify		No	No	No

Figure 22.6
Lock compatibility tables.
(a) A compatibility table for read/write locking scheme.
(b) A compatibility table for read/write/certify locking scheme.

1. Motivation

- In **standard 2PL**, if a transaction holds a **write lock**, **no other transaction can read or write** the item.
- This is restrictive because **reads get blocked** even when the item could safely be read as an older committed value.

Solution: Use **multiversion 2PL**:

- Maintain **two versions** of each item:
 1. **Committed version** – visible to other transactions for reading.
 2. **Uncommitted version** – used by the writing transaction for updates.

2. Locking Modes

Unlike standard 2PL (just read & write), MV2PL introduces **three modes**:

Lock Mode	Meaning
Read	Shared lock for reading committed versions
Write	Exclusive lock for writing a new version
Certify	Exclusive lock required at commit time to finalize changes

Compatibility Table:

Held / Requested	Read	Write	Certify
Read	Yes	Yes	No

Held / Requested	Read	Write	Certify
Write	Yes	No	No
Certify	No	No	No

Interpretation:

- **Read locks** don't block **write locks** on uncommitted versions.
- **Certify locks** are exclusive and block all other access; they are acquired **just before commit**.

3. How It Works

Step-by-step for a transaction T:

1. Writing an item X:

- T acquires a **write lock** on X.
- A **new uncommitted version X'** is created.
- Other transactions can **still read the committed version X** without blocking.

2. Reading an item X:

- A transaction always reads the **committed version**.
- This avoids reading uncommitted data → **no cascading aborts**.

3. Committing T:

- Before commit, T must acquire **certify locks** on all items it wrote.
- Certify locks are **exclusive** → T may have to wait for readers to finish.
- Once certify locks are acquired:
 - The committed version X is updated with the new value.
 - Temporary uncommitted version X' is discarded.
 - Certify locks are released.

4. Advantages

1. Increased concurrency:

- Reads can occur **concurrently with writes**, using the committed version.

2. No cascading aborts:

- Transactions read only committed versions.

3. Recoverability:

- Only committed data is visible to others.

5. Disadvantages / Caveats

1. Delayed commit:

- A transaction may have to wait for certify locks.

2. Deadlocks may still occur:

- Especially if **read locks are upgraded to write locks**.

3. Extra storage:

- Temporary uncommitted versions are created for each write.

✓ Intuition:

Think of it as **"staging changes"**:

- Writes happen in a **private workspace** (uncommitted version)
- Reads continue using the **public version**
- At commit, the workspace is **merged into the public version** after acquiring **certify locks**.

▼ OCC

Validation (Optimistic) Concurrency Control (OCC) based on your excerpt:

1. Core Idea

- Unlike locking or timestamp ordering, **OCC does not check for conflicts while a transaction executes**.
- Transactions proceed freely using **local copies** of data items.
- **Validation** occurs **at the end of the transaction**, before applying updates to the database.
- If validation passes → commit; otherwise → abort and restart.

Think of it as **"hope for the best, check at the end"**.

2. Phases of OCC

1. Read Phase:

- Transaction reads **committed values** from the database.
- Updates are made **only to local copies** (workspace).
- No checking for conflicts yet → very little overhead.

2. Validation Phase:

- Check whether the transaction can be applied **without violating serializability**.

- Uses:
 - `read_set` = items read by the transaction
 - `write_set` = items written by the transaction
 - Start and end times of the transaction phases
- Ensures the transaction **does not interfere** with other committed or currently validating transactions.

3. Write Phase:

- If validation passes → **apply updates to the database.**
- If validation fails → **discard local updates and restart** transaction.

3. Validation Rules

For a transaction `Ti` being validated against another transaction `Tj` (either committed or validating), one of these **three conditions must hold**:

1. **`Tj` completes its write phase before `Ti` starts its read phase**
 - No conflict possible since `Ti` reads after `Tj` has finished.
 2. **`Ti` starts its write phase after `Tj` completes its write phase, and $\text{read_set}(Ti) \cap \text{write_set}(Tj) = \emptyset$**
 - `Ti` writes after `Tj` writes, and `Ti` has not read anything `Tj` wrote → safe.
 3. **$\text{read_set}(Ti) \cap \text{write_set}(Tj) = \emptyset$ and $\text{write_set}(Ti) \cap \text{write_set}(Tj) = \emptyset$, and `Tj` completes its read phase before `Ti` completes its read phase**
 - `Ti` and `Tj` operate on disjoint items → safe.
- **If none of the conditions hold → `Ti` is aborted** and restarted.

Conditions are checked in order (1 → 2 → 3) to optimize efficiency.

4. Advantages of OCC

- Minimal overhead during transaction execution.
- Ideal for **low-conflict environments**:
 - Short transactions
 - Sparse overlapping of read/write sets
- Allows high concurrency compared to locking schemes.

5. Disadvantages / Caveats

- High interference → **many transactions may be aborted**, wasting work.
- Not suitable for **highly contended databases**.
- Requires maintaining **read/write sets and timestamps** for all active transactions.

✓ Intuition:

Think of OCC as a **sandbox approach**:

1. Work freely in your sandbox (local copies) → no blocking.
2. At the end, check whether merging your changes would break the rules (validation).
3. If it's safe → merge into the real world (commit).
4. If not → discard sandbox and try again (abort & restart).

▼ Granularity Level Considerations

Granularity Level Considerations for Locking is about **how big or small a "lockable unit" should be** in a database and how that choice affects **performance and concurrency**.

Let's break it down simply and intuitively 📌

🌱 1 What "Granularity" Means

Granularity = *size of the data item being locked*.

Type of Data Item	Example	Granularity Type
A single field	salary of one employee	Very fine
A record	entire row in a table	Fine
A disk block	multiple records in memory	Medium
A file	a whole table or index	Coarse
The entire database	all data	Very coarse

⚖️ 2 The Trade-Off (Concurrency vs Overhead)

Granularity Type	Effect on Concurrency	Effect on Overhead
Fine (small items)	✓ High concurrency (many transactions can work in parallel)	✗ High overhead (too many locks, more memory + time)
Coarse (large items)	✗ Low concurrency (fewer transactions can work together)	✓ Low overhead (fewer locks to manage)

💡 Example to Understand

Let's say we have a **block** that contains 10 **records**:

`[r1, r2, r3, ..., r10]`

Case 1: Coarse Granularity (Lock at Block Level)

- Transaction **T1** wants to modify record **r3**.
→ It must lock the **entire block**.
- Transaction **T2** wants to read record **r7** (same block).
→ It has to wait because the **whole block** is locked.

✓ Simple

✗ Poor concurrency

Case 2: Fine Granularity (Lock at Record Level)

- **T1** locks **r3**.
- **T2** locks **r7**.
→ Both proceed in parallel!

✓ Better concurrency

✗ More locks to manage → lock table grows → more CPU & memory cost.

⚙️ 3 So What's the Best Size?

👉 It **depends on the workload**:

Situation	Best Granularity
Transactions touch few records	Fine (record-level)
Transactions touch many records in one file	Coarse (file-level or block-level)

So a **static choice** is not always ideal

▼ Multiple Granularity Locking (MGL)

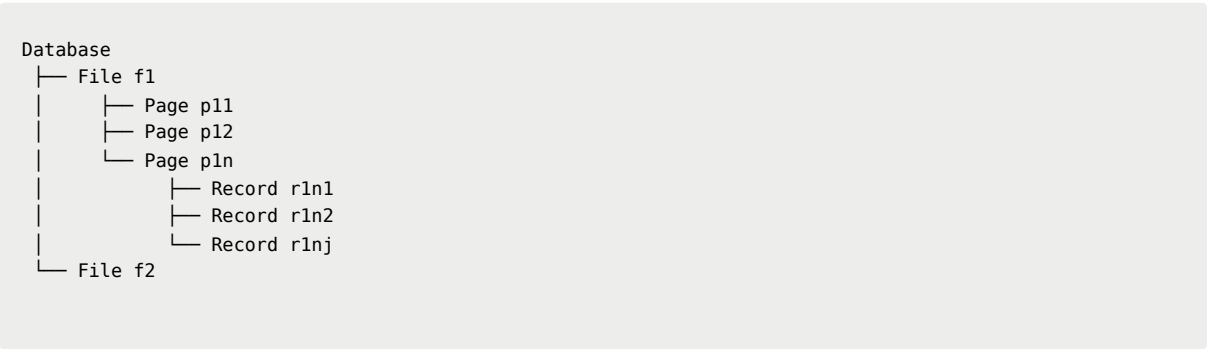
	IS	IX	S	SIX	X
IS	Yes	Yes	Yes	Yes	No
IX	Yes	Yes	No	No	No
S	Yes	No	Yes	No	No
SIX	Yes	No	No	No	No
X	No	No	No	No	No

Figure 22.8
Lock compatibility matrix for multiple granularity locking.

Let's go step-by-step so you see **why MGL is needed**, **what intention locks do**, and **how the MGL protocol works**.

1 The Core Problem

Imagine the hierarchy:



Situation 1 (Fine Lock First):

- **T2** locks **r1nj** (record-level shared lock).
- **T1** later wants to **update all of f1** → requests **exclusive lock on file f1**.

Now, to see if T1 can get the lock, the system must check:

“Are there any locks held on any record or page inside f1?”

That means scanning through possibly **millions of child nodes**.

→ **Expensive and inefficient**.

Situation 2 (Coarse Lock First):

- **T1** locks **f1** (exclusive).
- **T2** wants to read **r1nj**.
 - Easy check — T2 is blocked because parent **f1** is already exclusively locked.

So the problem is **how to efficiently detect conflicts** in both directions.

2 The Solution — Intention Locks

To make this efficient, transactions declare their *intentions* as they move down the tree.

Types of Intention Locks

Lock Type	Meaning
IS (Intention Shared)	“I intend to take shared locks on some child nodes.”

Lock Type	Meaning
IX (Intention Exclusive)	"I intend to take exclusive locks on some child nodes."
SIX (Shared + Intention Exclusive)	"I'm reading this node (shared) but will update some of its descendants."

So now, before a transaction takes a **record-level lock**, it must **place intention locks up the path** from the root → file → page → record.

3 Example with Intention Locks

Suppose:

- **T1** wants to update record `r111`.
- **T2** wants to update *all records on page* `p12`.
- **T3** wants to read record `r11j` and the entire file `f2`.

♦ Transaction T1

Wants to **update** (exclusive lock on `r111`).

1. Lock root (`db`) in **IX**
2. Lock `f1` in **IX**
3. Lock `p11` in **IX**
4. Lock `r111` in **X**

♦ Transaction T2

Wants to **update** all records on page `p12`.

1. Lock root (`db`) in **IX**
2. Lock `f1` in **IX**
3. Lock `p12` in **X**

♦ Transaction T3

Wants to **read** record `r11j` (shared) and file `f2`.

1. Lock root (`db`) in **IS**
2. Lock `f1` in **IS**
3. Lock `p11` in **IS**
4. Lock `r11j` in **S**

5. Lock **f2** in **S**

4 Lock Compatibility (Simplified)

	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	✗
IX	✓	✓	✗	✗	✗
S	✓	✗	✓	✗	✗
SIX	✓	✗	✗	✗	✗
X	✗	✗	✗	✗	✗

✓ means compatible, ✗ means conflict.

So:

- IS and IX can coexist.
- But S conflicts with IX, SIX, or X (to prevent dirty reads).

5 MGL Protocol Rules (Intuitive Summary)

1. **Follow the compatibility matrix** (no conflicting locks).
2. **Lock the root first** (always start from the top).
3. To lock a node:
 - If you want **S** or **IS**, parent must be locked in **IS** or **IX**.
 - If you want **X**, **IX**, or **SIX**, parent must be in **IX** or **SIX**.
4. **Follow 2-Phase Locking** (acquire all before releasing any).
5. You can only **unlock a node when none of its children are locked**.

6 Why It Works

- **Efficient conflict checking:**

Because every transaction leaves "intention marks" up the tree, the lock manager doesn't have to scan all descendants.

- **Flexible granularity:**

Some transactions can lock **records**, others can lock **entire files**, safely.

- **Maintains serializability:**

Still follows 2PL → produces conflict-serializable schedules.

✓ 7 Example Intuition Summary

Transaction	Operation	Locks Acquired	Benefit
T1	Update 2 records	IX + X on 2 records	Fine-grained control
T2	Update a whole page	IX + X on that page	Efficient bulk update
T3	Read one record + one file	IS + S	Safe and parallel reads